

INFORME FINAL DE INGENIERÍA

Proyecto 15: Prototipo de Reservas o Turnos Médicos Cloud (Lite)

Desarrollado por: * Carlos Fuentes, Ingeniero Cloud Jr.

- Luis Gerardo Puentes Lopez, Desarrollador Fullstack Jr.

Institución: Escuela Superior de Innovación y Tecnología (ESIT)

Carrera: Técnico en Servicios en la Nube

Fecha de Entrega: 31 de enero de 2026

1. RESUMEN EJECUTIVO

Este proyecto presenta el desarrollo de un prototipo funcional para la gestión de turnos médicos en la nube, diseñado bajo el modelo **SaaS (Software as a Service)**. La solución permite la administración automatizada de citas para la "Clínica Integral Santa Salud (CISS)", resolviendo problemas críticos de concurrencia y disponibilidad de recursos humanos en tiempo real mediante una arquitectura escalable y eficiente.

2. ARQUITECTURA DEL SISTEMA

El sistema utiliza una arquitectura **Serverless** integrada en el ecosistema de Google Cloud, garantizando alta disponibilidad sin gestión de servidores físicos:

- **Frontend:** Interfaz web responsiva (HTML5/JavaScript) que se comunica de forma asíncrona con el servidor.
 - **Backend (Engine):** Lógica procesada en Google Apps Script, encargada de las reglas de negocio y validación.
 - **Base de Datos:** Google Sheets, utilizado como repositorio de datos estructurados para el almacenamiento de registros históricos.
 - **Security & Concurrency:** Implementación de `LockService` para prevenir colisiones de escritura y asegurar la integridad de los datos en entornos multiusuario.
-

3. GUÍA DE OPERACIÓN (USUARIO FINAL)

1. **Ingreso:** El administrador accede mediante una interfaz de login (u: admin / p: 12345).
 2. **Agendado de Citas:** Se registran los datos del paciente. El sistema valida automáticamente la disponibilidad según la especialidad.
 3. **Monitoreo:** El listado principal permite filtrar citas por fecha, especialidad o nombre para un control efectivo de la agenda diaria.
 4. **Gestión de Estados:** *  **(Completada):** Cierra el ciclo de la cita.
 - o  **(Cancelada):** Libera el cupo en el servidor de forma inmediata.
 - o **NA (No asistió):** Registra la incidencia para historial médico.
 - o  **(Reagendar):** Permite modificar fecha y hora; la lógica de ingeniería valida que el nuevo horario tenga cupos libres antes de confirmar.
-

4. LISTADO DE FUNCIONES OPERATIVAS

Backend (Lógica de Ingeniería en Código.gs)

- `agendarCita(datos)`: Procesa el registro y dispara el servicio de correo SMTP.
- `hayChoque(fecha, hora, especialidad, idExcluir)`: Algoritmo de validación que normaliza formatos de tiempo y gestiona la exclusión de IDs en procesos de edición.
- `modificarCita(id, campos)`: Actualización directa de la persistencia de datos en la nube.
- `editarCita(id, nuevosDatos)`: Valida la disponibilidad en el nuevo bloque horario antes de ejecutar el cambio.

Frontend (Lógica de Interfaz en index.html)

- `agendar()`: Captura la entrada del usuario y gestiona los estados de espera y éxito.
 - `cargarCitas()`: Actualización asíncrona de la tabla de registros.
 - `mostrarCitas(citas)`: Construcción dinámica del DOM para la visualización de la agenda.
 - `estado(id, nuevoEstado)`: Puerta de enlace para la actualización de estatus.
-

5. RESPALDO COMPLETO DE CÓDIGO (BACKUP)

Archivo: Código.gs

JavaScript

```
/****** CONFIGURACIÓN DE NUBE *****/  
const SPREADSHEET_ID = "1cbRC8rujJ4kvVDrOTQ2Yx-v6ZR0gu1ZDHV-bJXAdfWA";
```

```

const SHEET_NAME = "CitasAgendadas";

const ESPECIALIDADES = {
  "Medicina General": 3,
  "Medicina Interna": 1,
  "Pediatría": 1,
  "Ginecología": 1
};

const ESTADOS = { AGENDADA: "Agendada", CANCELADA: "Cancelada", COMPLETADA:
"Completada", NO_ASISTIO: "No asistio" };

/***** VALIDACIÓN DE DISPONIBILIDAD CLOUD *****/
function hayChoque(fecha, hora, especialidad, idExcluir = null) {
  const sheet = getSheet();
  const data = sheet.getDataRange().getValues();
  if (data.length <= 1) return false;

  const capacidad = ESPECIALIDADES[especialidad] || 1;
  let ocupadas = 0;

  let fBusqueda = String(fecha).trim();
  if (fBusqueda.includes("/")) {
    const partes = fBusqueda.split("/");
    fBusqueda = `${partes[2]}-${partes[1]}-${partes[0]}`;
  }
  const hBusqueda = String(hora).trim().substring(0, 5);

  for (let i = 1; i < data.length; i++) {
    const row = data[i];
    const idFila = String(row[7] || "").trim();
    if (idExcluir && idFila === idExcluir) continue;

    const espCelda = String(row[5] || "").trim();
    const estCelda = String(row[6] || "").trim();
    if (estCelda !== ESTADOS.AGENDADA) continue;

    const fString = (row[1] instanceof Date) ? Utilities.formatDate(row[1],
Session.getScriptTimeZone(), "yyyy-MM-dd") : String(row[1]).trim();
    let hString = (row[2] instanceof Date) ? Utilities.formatDate(row[2],
Session.getScriptTimeZone(), "HH:mm") : String(row[2]).trim().substring(0,
5);

    if (fString === fBusqueda && hString === hBusqueda && espCelda ===
especialidad) {
      ocupadas++;
    }
  }
  return ocupadas >= capacidad;
}

/***** REAGENDADO DE TURNOS *****/
function editarCita(id, nuevosDatos) {
  const lock = LockService.getScriptLock();
  lock.waitLock(5000);
  try {
    const sh = getSheet();

```

```

const data = sh.getDataRange().getValues();
const idBuscado = String(id).trim();
let filaCita = -1;
let especialidad = "";

for (let i = 1; i < data.length; i++) {
  if (String(data[i][7]).trim() === idBuscado) {
    filaCita = i + 1;
    especialidad = data[i][5];
    break;
  }
}
if (filaCita === -1) throw new Error("Cita no encontrada");

if (hayChoque(nuevosDatos.fecha, nuevosDatos.hora, especialidad,
idBuscado)) {
  throw new Error("Cupo lleno en el nuevo horario seleccionado.");
}

sh.getRange(filaCita, 2).setValue(nuevosDatos.fecha);
sh.getRange(filaCita, 3).setValue(nuevosDatos.hora);
return true;
} finally {
  lock.releaseLock();
}
}

/***** MODIFICAR ESTADO *****/
function modificarCita(id, campos) {
  const sh = getSheet();
  const data = sh.getDataRange().getValues();
  const idBuscado = String(id).trim();
  for (let i = 1; i < data.length; i++) {
    if (String(data[i][7] || "").trim() === idBuscado) {
      if (campos.estado) sh.getRange(i + 1, 7).setValue(campos.estado);
      return { success: true };
    }
  }
  return { success: false };
}

```

Firmado por:

Carlos Fuentes	Luis Gerardo Puentes Lopez
Ingeniero Cloud Jr.	Desarrollador Fullstack Jr.